Big Data Analysis of Amazon Book Reviews Using Apache Spark

Deniz Topal
Artificial Intelligence and Data Engineering
Istanbul Technical University
Istanbul, Turkey
topald20@itu.edu.tr

Furkan Yasir Göksu
Artificial Intelligence and Data Engineering
Istanbul Technical University
Istanbul, Turkey
goksu20@itu.edu.tr

Enes Karabulut
Artificial Intelligence and Data Engineering
Istanbul Technical University
Istanbul, Turkey
karabulute23@itu.edu.tr

***Abstract*—This paper presents a big data analytics pipeline for processing and analyzing more than three million Amazon book reviews using Apache Spark. Our work integrates data cleaning, exploratory data analysis, sentiment classification, and two recommendation approaches. First, we conduct extensive data analysis and transform the raw CSV files into a structured, machine-learning-friendly format. Second, we explore the dataset by visualizing popular books, investigating correlations between price and ratings, and examining user behavior over time. Third, we apply sentiment analysis on review text to distinguish positive from negative feedback. Finally, we introduce both collaborative filtering and content-based recommendation methods, followed by a thorough evaluation of predictive accuracy. Our experimental results highlight the utility of big data methods in deriving meaningful insights and generating personalized suggestions for readers.**

***Index Terms*—Big Data Analytics, Apache Spark, Recommendation Systems, Sentiment Analysis, Exploratory Data Analysis**

I. Introduction

This paper presents the results of our term project for the Big Data Analytics course, which involved analyzing over three million Amazon book reviews using advanced big data techniques. The primary goal of our project was to explore trends, patterns, and user behaviors in the dataset and to build an efficient recommendation system based on user preferences and book characteristics.

We began by processing two large datasets: one containing book metadata such as titles, authors, categories, and descriptions, and another with detailed user review data, including ratings, timestamps, and review text. These datasets provided a rich source of information to understand user engagement and book popularity trends.

Our main objectives include:

- **Data Analysis:** Integrate and clean two main CSV files (one for book metadata, one for user ratings).
- **Exploratory Data Analysis (EDA):** Examine user behavior, popular titles, yearly trends, and price-rating correlations.
- **Sentiment Analysis:** Classify textual reviews as positive or negative to uncover hidden attitudes.
- **Recommendation System:** Build collaborative filtering and content-based filtering models to recommend books effectively.

The remainder of this paper is organized as follows. Section II presents our methodology in several subsections: (1) data analysis, (2) EDA, (3) sentiment, (4) recommendation system, and (5) evaluation. We conclude with a summary of our findings and directions for future research.

II. Methodology

We define our methodological pipeline in five major steps. Each step leverages Apache Spark for distributed data processing and draws on Python libraries for data visualization and modeling.

A. Data Analysis

We base our approach on two CSV files:

- `books_info.csv`: Contains essential metadata about each book, such as title, authors, categories, published date, and description.
- `books_rating.csv`: Includes user-specific rating information, such as review scores (1–5), review text, user identifiers, and timestamps.

Data Cleaning: We drop unused columns (e.g., *previewLink*, *infoLink*) and convert textual fields like *description* to lower case. We also trim whitespace from columns to normalize strings. Null or empty rows in critical fields (e.g., *Title*, *User_id*) are removed to ensure data integrity.

Integration and Indexing: We join `books_info.csv` and `books_rating.csv` on the *Title* attribute. Using Spark's `StringIndexer`, we transform *Title* into a numeric *Book_Id* and similarly map *User_id* into an integer index. This step is crucial for machine learning algorithms like ALS (Alternating Least Squares) and for efficient group-by operations.

Schema Validation: We enforce a well-defined schema with Spark's `StructType` and `StructField`, ensuring each column is assigned the correct data type (e.g., string, float, or integer). This practice reduces the likelihood of runtime errors and fosters consistency in subsequent analysis.

B. Exploratory Data Analysis (EDA)

After cleaning the data, we performed Exploratory Data Analysis (EDA) to better understand the dataset and identify key patterns.

Popular Books and User Engagement: To determine which books were most popular, we counted how many times each book received a perfect score (5.0). This helped us identify books that users highly favored. Additionally, we grouped reviews by year (*review/time*) to observe trends in user activity. This analysis showed certain years with increased numbers of reviews, which could indicate either a rise in the platform's popularity or the release of widely discussed books. Figure 1 shows the top 10 most-read books based on user reviews.

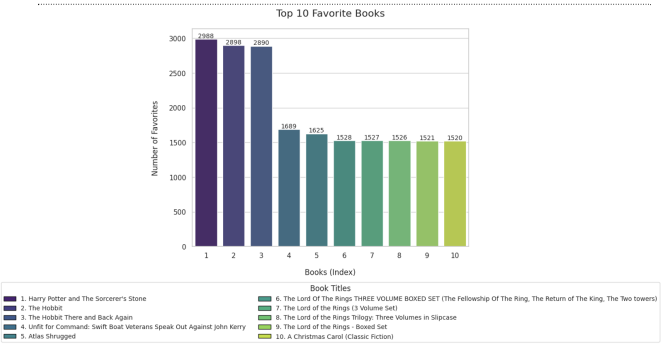

Fig. 1. Top 10 Most Read Books Based on User Engagement

Price vs. Rating Correlation: We explored the relationship between book prices and user ratings by dividing the *Price* column into ranges (e.g., {0–10, 10–20, 20–30}) and calculating the average rating within each range. Additionally, we calculated the Pearson correlation coefficient to measure the strength of this relationship. Our results showed a weak correlation, meaning higher-priced books do not necessarily receive better ratings. This suggests that factors like the author's reputation or book content may have a greater influence on user ratings than price.

Yearly Price Averages: To study how book prices have changed over time, we calculated the average price of books reviewed each year. This analysis helps identify trends in pricing strategies across the years, particularly for specialized or technical books. Figure 2 presents the average book prices by year, showing fluctuations and trends in how books are priced over time.

Through EDA, we gained important insights into user preferences, book pricing, and activity patterns. These findings form the basis for more advanced analyses, such as sentiment analysis and recommendation systems, discussed in the following sections.

C. Sentiment Analysis

Sentiment analysis was performed to extract emotional insights from the *review/text* column, adding a qualitative layer to the dataset. This step helps understand user satisfaction

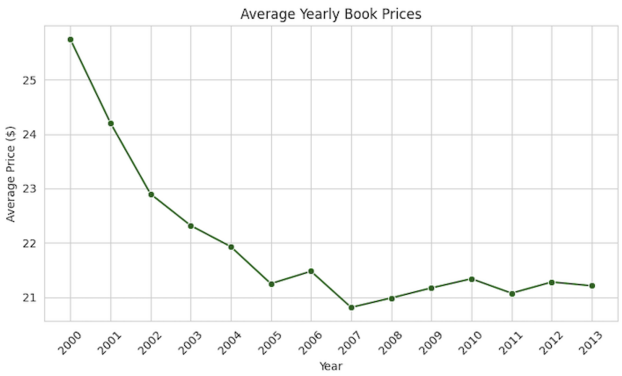

Fig. 2. Yearly Average Book Prices

and highlights feedback beyond the numeric ratings. Using Spark NLP, we implemented a pipeline to classify reviews as either *positive* or *negative*, allowing us to analyze trends in user sentiment across different books and categories.

The pipeline included several components:

- **DocumentAssembler:** Converts the raw text reviews into a document format required by Spark NLP.
- **Tokenizer:** Breaks the document into individual tokens or words.
- **Normalizer:** Cleans the tokens by removing unwanted symbols, punctuation, and special characters.
- **ViveknSentimentModel:** A pretrained model that analyzes the text and assigns a sentiment label (*positive* or *negative*) based on linguistic features.
- **Finisher:** Extracts the final sentiment labels for storage and visualization.

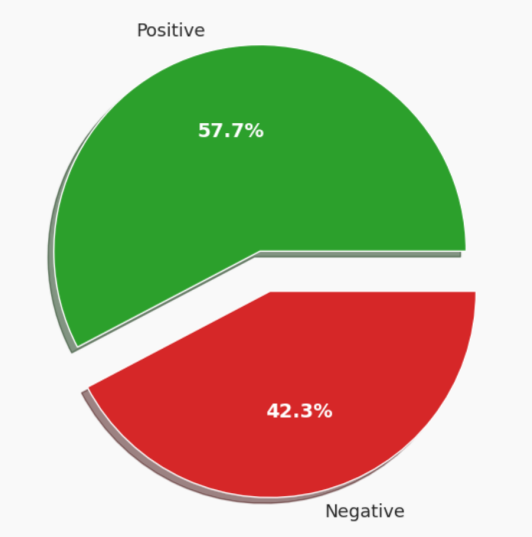

Fig. 3. Distribution of Reviews by Sentiment

After applying the pipeline, we found that approximately 57.7% of the reviews were classified as positive, while 42.3% were negative. This slight skew toward positive reviews indicates that most users were satisfied with their purchases, although a significant proportion also expressed critical opinions.

To explore sentiment trends further, we analyzed the categories with the highest positive review rates. Using the sentiment-labeled reviews, we joined the dataset with book categories and identified which genres were most positively received by users. This analysis revealed that categories such as fiction, self-help, and biographies tended to garner the most favorable feedback, reflecting user satisfaction within these genres.

To visualize this insight, we created a word cloud (Figure 4) representing the categories with the highest positive review rates. Larger words in the word cloud indicate categories with more positive reviews, highlighting their popularity among users.

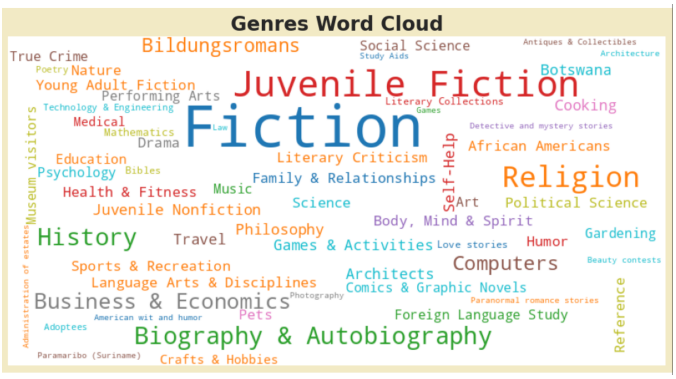

Fig. 4. Word Cloud of Categories with the Highest Positive Review Rates

1) Collaborative Filtering (CF): Collaborative Filtering (CF) is a recommendation technique that predicts user preferences by analyzing past interactions between users and items. In our project, we implemented CF using the Alternating Least Squares (ALS) algorithm provided by Apache Spark MLlib.

Spark Implementation: ALS

The ALS algorithm factorizes the user-item rating matrix R into two lower-dimensional matrices:

$$R \approx U \times P \tag{1}$$

where R represents the user-item matrix, U denotes the user latent factors, and P denotes the item latent factors. The objective of ALS is to minimize the mean squared error by iteratively updating U and P to approximate the original matrix.

Data Preparation for ALS

To prepare the dataset, we selected relevant columns such as user identifiers, book identifiers, and review scores. The dataset was split into training (80%) and testing (20%) sets. One of the challenges encountered during this process was the *cold start problem*, which occurs when users or items have insufficient data to generate reliable recommendations. To address this, Spark's built-in cold-start strategy was employed to handle missing predictions.

User id	Item id	Rating
1	a	1
1	b	2
2	a	3
2	c	2
3	c	5

Data

	item a	item b	item c
user 1	1	2	nan
user 2	3	nan	2
user 3	nan	nan	5

Utility Matrix

Fig. 5. ALS Matrix Factorization Process

Predictions and Recommendations

Once the model was trained, predictions were generated to estimate user ratings for books they had not reviewed. The predicted rating $\hat{r}_{ui}$ for a user-item pair was computed as:

$$\hat{r}_{ui} = U_u \cdot P_i \tag{2}$$

The top book recommendations were provided to users based on the highest predicted ratings.

TABLE I
TOP BOOK RECOMMENDATIONS

Book_Id	User_id	Prediction	Title
2573	0	5.8810754	Men We Cherish
3818	0	5.928539	The ADHD Affected...
4329	0	6.1241674	The Machinery of ...

Evaluation

The performance of the ALS model was evaluated using the Root Mean Squared Error (RMSE) metric, which measures the deviation between actual and predicted ratings. RMSE is calculated using the following formula:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\text{review/score}_i - \text{prediction}_i)^2} \tag{3}$$

In our experiments, the model achieved an RMSE of 1.302, indicating a reasonable level of accuracy. The error distribution revealed that most predictions were within acceptable error margins, confirming the model's effectiveness.

The collaborative filtering approach successfully generated personalized book recommendations by leveraging user interactions. Future improvements could include:

- Fine-tuning ALS hyperparameters to enhance model accuracy.
- Combining CF with content-based filtering for a hybrid approach.
- Addressing the cold-start problem by incorporating implicit feedback.

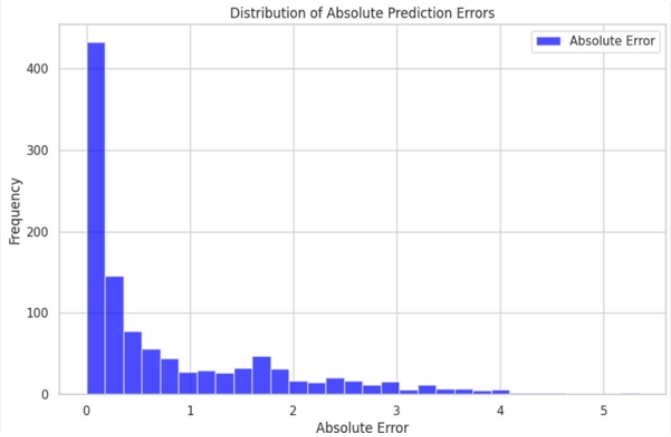

Fig. 6. Distribution of Absolute Prediction Errors

2) Content-Based Filtering (CBF): Content-Based Filtering (CBF) is a recommendation approach that suggests books to users based on their inherent features and the user's preferences. In our project, we implemented a robust and scalable pipeline using Apache Spark and Spark NLP to analyze book metadata and textual descriptions. By processing key features such as book descriptions, authors, and categories, we created meaningful feature vectors that encapsulate the semantic and structural characteristics of each book. This approach allows the system to recommend books with similar attributes to those the user has previously interacted with, making it particularly effective for handling cold-start problems for items, as it does not rely on prior user interaction data.

The core idea of our CBF implementation involves constructing personalized user vectors by aggregating and weighting the feature vectors of books based on the user's ratings. These user vectors are then compared with all book vectors using similarity metrics such as cosine similarity and approximate nearest neighbors to identify the most relevant recommendations. This methodology ensures precise and explainable recommendations while leveraging metadata-driven insights to align with user preferences. The pipeline is designed to scale efficiently, processing large datasets and providing recommendations with minimal latency.

1. Feature Extraction

To represent books as numerical vectors, we extracted and encoded their key attributes:

- **Description Vector:** Book descriptions were processed using Spark NLP and then converted into numerical vectors using the Word2Vec algorithm. Word2Vec creates dense vector embeddings that capture the semantic meaning of the text, ensuring that similar descriptions yield similar vectors.
- **Author Embedding:** Authors were treated as textual data and encoded using Word2Vec. This approach captures the stylistic or thematic similarities between different authors, providing a meaningful representation of their contributions.
- **Category Encoding:** Each book's categories (e.g., fiction, self-help) were transformed into multi-hot vectors. A multi-hot vector indicates the genres a book belongs to, allowing the system to factor in genre-based similarities.

These feature vectors were assembled using Spark's `VectorAssembler`, which combines the individual components into a unified feature vector for each book.

2. Text Preprocessing Pipeline

The book descriptions underwent a detailed preprocessing pipeline using Spark NLP, as shown in Figure 7. This ensured that the text data was clean, normalized, and ready for feature extraction. The pipeline included:

- **Document Assembler:** Converts raw text into a structured document format required for NLP processing.
- **Tokenizer:** Breaks the document into individual tokens or words, enabling further processing.
- **Normalizer:** Cleans tokens by removing special characters, symbols, and punctuation to standardize the text.
- **StopWordsCleaner:** Removes commonly used stop words (e.g., "and," "the") to focus on meaningful terms.
- **Lemmatizer:** Converts tokens to their base forms (e.g., "running" to "run") to reduce variability in the text.

The pipeline output was fed into the Word2Vec algorithm, which produced dense numerical vectors for each book description. These vectors encapsulated the semantic essence of the text, enabling effective similarity computation.

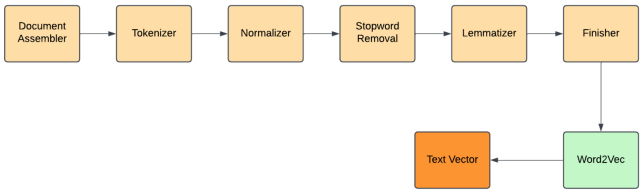

Fig. 7. NLP Pipeline for Preprocessing Text Features

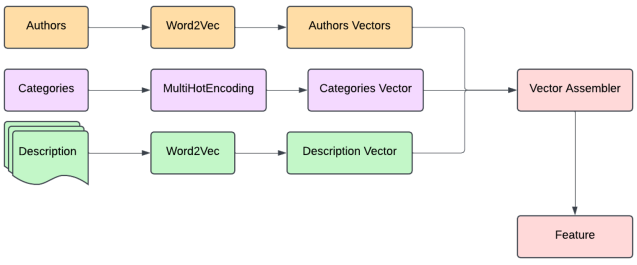

Fig. 8. Feature Extraction Process

3. User Profile Vector

To create personalized recommendations, we constructed a user profile vector. This vector represents the user's preferences by aggregating the feature vectors of the books they have rated. The aggregation process accounts for the user's ratings to weigh each book's contribution:

$$v_i^{weighted} = v_i \cdot \frac{r_i}{\text{Total Ratings}}$$

where v_i is the feature vector of book i, r_i is the user's rating for book i, and Total Ratings is the sum of all ratings provided by the user.

The user profile vector is then computed as:

$$u_{feature} = \sum_{i=1}^{n} v_i^{weighted}$$

This approach ensures that highly rated books have a stronger influence on the user's profile, capturing their preferences effectively.

4. Similarity Computation

To find books similar to the user's preferences, we calculated the similarity between the user vector and the feature vectors of all books. Two approaches were used: The similarity between the user profile vector and the feature vectors of all books was calculated using two approaches:

- **Cosine Similarity:** Measures the cosine of the angle between the user vector and book vectors. A higher cosine similarity indicates greater alignment between the user's preferences and the book's features.

$$\text{Cosine Similarity} = \frac{u_{feature} \cdot v}{\|u_{feature}\| \|v\|}$$

- **Approximate Nearest Neighbors (ANN):** To improve computational efficiency, we used Spark's `BucketedRandomProjectionLSH`, an ANN method. This approach approximates nearest neighbors in high-dimensional spaces, enabling scalable similarity computations.

5. Recommendation Generation

The recommendation process involved the following steps:

1) Compute the user profile vector $u_{feature}$ by aggregating and weighting the feature vectors of rated books.
2) Use approximate nearest neighbors to identify the top k books with the highest similarity to $u_{feature}$.
3) Rank the books by their similarity scores and present the top recommendations to the user.

Advantages of Content-Based Filtering

- **Cold-Start for Items:** CBF can recommend newly added books based on their metadata without requiring prior user interactions.
- **Explainability:** Recommendations can be justified based on content similarity (e.g., similar descriptions, authors, or genres).

Future Improvements Future work may involve integrating advanced embedding techniques like BERT or leveraging hybrid approaches that combine CBF with collaborative filtering to further enhance recommendation quality.

III. Conclusion

In this work, we explored over three million Amazon book reviews to reveal user behavior patterns, sentiment tendencies, and opportunities for personalized recommendations. By harnessing Apache Spark, we efficiently cleaned and processed large CSV files, performed comprehensive *EDA*, and built two categories of recommenders (*CF* and *CBF*). Our sentiment analysis using Spark NLP uncovered a near 60%-40% split between positive and negative feedback, providing deeper insight into user satisfaction than ratings alone.

Future research directions include hyperparameter optimization for ALS and Word2Vec, adoption of advanced language models like BERT for improved semantic embeddings, and a hybrid approach combining both CF and CBF predictions. Additional user-level experiments or real-time incremental updates could further enhance recommendation quality.

Acknowledgments

We thank Istanbul Technical University for providing the infrastructure and environment to execute large-scale analytics on Google Cloud, as well as the guidance received from the Big Data Analytics course instructors.

article

url

References

[1] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*, 2012.

[2] X. Amatriain and J. Basilico, "Building Effective Recommendation Systems," *IEEE Data Engineering Bulletin*, vol. 36, no. 4, pp. 3–9, Dec. 2013.

[3] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," *arXiv preprint arXiv:1301.3781*, 2013.

[4] Spark NLP, "Sentiment Analysis Vivekn," https://sparknlp.org/2021/11/22/sentiment$_vivekn_en.html, accessed Jan. 2025.

[5] Apache Spark, "pyspark.ml.recommendation.ALS," https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html, accessed Jan. 2025.

[6] B. Borges, "Recommender System using ALS in PySpark," https://medium.com/@brunoborges$_38708/recommender-system-using-als-in-pyspark-10329e1d1ee1, accessed Jan. 2025.

[7] Analytics Vidhya, "Model-based Recommendation System with Matrix Factorization: ALS Model and the Math Behind," https://medium.com/analytics-vidhya/model-based-recommendation-system-with-matrix-factorization-als-model-and-the-math-behind-fdce8b2ffe6d, accessed Jan. 2025.

[8] S. Watsford, "Gentle ALS," https://sophwats.github.io/2018-04-05-gentle-als.html, accessed Jan. 2025.

[9] Spark NLP, "Lemma Annotator," https://sparknlp.org/2021/11/22/lemma$_antbc_en.html, accessed Jan. 2025.